Программная реализация симплекс-метода
Вариант 17

В программе реализованы автоматическое формирование вспомогательной задачи из исходных данных, вывод симплекс-таблицы, выбор разрешающего столбца и строки, пересчёт таблицы, выполнение первой и второй фаз и получение итогового решения

1. Класс SimplexSolver и init

Создаём объект решателя, сохраняет исходную целевую функцию и ограничения, проверяет направление оптимизации и типы переменных. По умолчанию переменные считаются неотрицательными



```python
from fractions import Fraction


class SimplexSolver:
    def __init__(
        self,
        objective,
        constraints,
        sense="max",
        variable_signs=None,
    ):
        self.input_objective = [
            Fraction(value) for value in objective
        ]

        self.input_constraints = [
            (
                [Fraction(value) for value in coefficients],
                sign,
                Fraction(rhs),
            )
            for coefficients, sign, rhs in constraints
        ]

        self.sense = sense.lower()

        if self.sense not in ("max", "min"):
            raise ValueError(
                "Направление должно быть 'max' или 'min'"
            )

        self.variable_count = len(objective)

        if variable_signs is None:
            variable_signs = [
                "nonnegative"
                for _ in range(self.variable_count)
            ]

        if len(variable_signs) != self.variable_count:
            raise ValueError(
                "Количество типов переменных не совпадает "
                "с количеством переменных"
            )

        for sign in variable_signs:
            if sign not in ("nonnegative", "free"):
                raise ValueError(
                    "Тип переменной должен быть "
                    "'nonnegative' или 'free'"
                )

        self.variable_signs = variable_signs

        self.objective = []
        self.constraints = []

        self.variables = []
        self.var_map = {}

        self.table = []
        self.basis = []
        self.nonbasis = []
        self.artificial = set()
```

2. Функция print_table

Выводит текущую симплекс-таблицу: небазисные переменные, базисные переменные, свободные члены и строку целевой функции

```python
    def print_table(self, title, objective_name="W"):
        print(f"\n{title}")

        print(f"{'':>8}", end="")

        for variable in self.nonbasis:
            print(f"{variable:>10}", end="")

        print(f"{'b':>10}")

        for i in range(len(self.table) - 1):
            print(f"{self.basis[i]:>8}", end="")

            for value in self.table[i]:
                print(f"{str(value):>10}", end="")

            print()

        print(f"{objective_name:>8}", end="")

        for value in self.table[-1]:
            print(f"{str(value):>10}", end="")

        print()
```

3. Функция transform_variables

Преобразуем свободные переменные. Если переменная свободна по знаку, она заменяется разностью двух неотрицательных переменных: x = x_plus - x_minus. После этого пересчитываются коэффициенты целевой функции и ограничений

```python

    def transform_variables(self):
        self.variables = []
        self.var_map = {}
        self.objective = []

        for i in range(self.variable_count):
            name = f"x{i + 1}"
            coefficient = self.input_objective[i]

            if self.variable_signs[i] == "nonnegative":
                self.variables.append(name)
                self.objective.append(coefficient)

                self.var_map[name] = [
                    (name, Fraction(1))
                ]

            else:
                plus_name = f"x{i + 1}_plus"
                minus_name = f"x{i + 1}_minus"

                self.variables.append(plus_name)
                self.variables.append(minus_name)

                self.objective.append(coefficient)
                self.objective.append(-coefficient)

                self.var_map[name] = [
                    (plus_name, Fraction(1)),
                    (minus_name, Fraction(-1)),
                ]

        indexes = {
            name: i
            for i, name in enumerate(self.variables)
        }

        new_constraints = []

        for coefficients, sign, rhs in self.input_constraints:
            new_coefficients = [
                Fraction(0)
                for _ in self.variables
            ]

            for i, coefficient in enumerate(coefficients):
                name = f"x{i + 1}"

                for new_name, multiplier in self.var_map[name]:
                    index = indexes[new_name]

                    new_coefficients[index] += (
                        coefficient * multiplier
                    )

            new_constraints.append(
                (new_coefficients, sign, rhs)
            )

        self.constraints = new_constraints
```

4. Функция normalize

Обрабатывает ограничения с отрицательной правой частью. Если b < 0, все коэффициенты и правая часть умножаются на -1, а знаки <= и >= меняются местами

```python
    def normalize(self):
        new_constraints = []

        for coefficients, sign, rhs in self.constraints:
            coefficients = coefficients[:]

            if rhs < 0:
                coefficients = [
                    -value for value in coefficients
                ]

                rhs = -rhs

                if sign == "<=":
                    sign = ">="
                elif sign == ">=":
                    sign = "<="
                elif sign == "=":
                    sign = "="
                else:
                    raise ValueError(
                        f"Неизвестный знак ограничения: {sign}"
                    )

            if sign not in ("<=", ">=", "="):
                raise ValueError(
                    f"Неизвестный знак ограничения: {sign}"
                )

            new_constraints.append(
                (coefficients, sign, rhs)
            )

        self.constraints = new_constraints
```

5. Функция prepare_problem

Последовательно запускает преобразование свободных переменных и нормализацию ограничений

```python
    def prepare_problem(self):
        self.transform_variables()
        self.normalize()
```

6. Функция build_phase_one

Строит первую фазу симплекс-метода. Добавляет дополнительные переменные для неравенств, создаёт искусственные переменные, формирует начальный базис и строку вспомогательной целевой функции W

```python
def build_phase_one(self):
    original_variables = self.variables[:]

    next_number = self.variable_count + 1

    used_names = set(original_variables)

    def new_variable():
        nonlocal next_number

        while f"x{next_number}" in used_names:
            next_number += 1

        name = f"x{next_number}"
        used_names.add(name)
        next_number += 1

        return name

    additional_variables = []
    additional_for_row = {}
```

```
        for i, (_, sign, _) in enumerate(self.constraints):
            if sign == "<=":
                name = new_variable()

                additional_variables.append(name)
                additional_for_row[i] = (
                    name,
                    Fraction(1),
                )

            elif sign == ">=":
                name = new_variable()

                additional_variables.append(name)
                additional_for_row[i] = (
                    name,
                    Fraction(-1),
                )

        self.nonbasis = (
            original_variables + additional_variables
        )

        indexes = {
            name: i
            for i, name in enumerate(self.nonbasis)
        }

        rows = []
```

```
        for i, (coefficients, _, rhs) in enumerate(
            self.constraints
        ):
            row = coefficients[:]

            row += [
                Fraction(0)
                for _ in additional_variables
            ]

            if i in additional_for_row:
                name, value = additional_for_row[i]
                row[indexes[name]] = value

            row.append(rhs)
            rows.append(row)

        self.basis = []
        self.artificial = set()

        for _ in self.constraints:
            name = new_variable()

            self.artificial.add(name)
            self.basis.append(name)

        objective_row = [
            Fraction(0)
            for _ in range(len(self.nonbasis) + 1)
        ]

        for row in rows:
            for j in range(len(row)):
                objective_row[j] -= row[j]

        self.table = rows + [objective_row]
```

7. Функция find_entering_column

Ищет разрешающий столбец по строке целевой функции. Если отрицательных коэффициентов больше нет, текущая таблица является оптимальной

```
    def find_entering_column(self):
        objective_row = self.table[-1][:-1]

        min_value = min(objective_row)

        if min_value >= 0:
            return None

        return objective_row.index(min_value)
```

8. Функция find_leaving_row

Выбирает разрешающую строку по минимальному положительному отношению правой части к положительному элементу разрешающего столбца

```
    def find_leaving_row(self, column):
        best_row = None
        best_ratio = None

        for i in range(len(self.table) - 1):
            coefficient = self.table[i][column]
            rhs = self.table[i][-1]

            if coefficient > 0:
                ratio = rhs / coefficient

                if (
                    best_ratio is None
                    or ratio < best_ratio
                ):
                    best_ratio = ratio
                    best_row = i

        return best_row
```

9. Функция pivot

Выполняет пересчёт симплекс-таблицы относительно разрешающего элемента и меняет базисную и небазисную переменные местами

```
    def pivot(self, row, column):
        pivot_element = self.table[row][column]
        old_pivot_row = self.table[row][:]
        leaving_variable = self.basis[row]
        entering_variable = self.nonbasis[column]
        print(
            f"\nРазрешающий столбец: "
            f"{entering_variable}"
        )
        print(
            f"Разрешающая строка: "
            f"{leaving_variable}"
        )
        print(
            f"\nВходит в базис: "
            f"{entering_variable}"
        )
        print(
            f"Выходит из базиса: "
            f"{leaving_variable}"
        )
        print(
            f"Разрешающий элемент: "
            f"{pivot_element}"
        )
        new_pivot_row = [
            Fraction(0)
            for _ in self.table[row]
        ]
        for j in range(len(self.nonbasis)):
            if j == column:
                new_pivot_row[j] = (
                    Fraction(1) / pivot_element
                )
            else:
                new_pivot_row[j] = (
                    old_pivot_row[j] / pivot_element
                )
        new_pivot_row[-1] = (
            old_pivot_row[-1] / pivot_element
        )
        for i in range(len(self.table)):
            if i == row:
                continue
            old_row = self.table[i][:]
            coefficient = old_row[column]
            for j in range(len(self.nonbasis)):
                if j == column:
                    self.table[i][j] = (
                        -coefficient / pivot_element
                    )
                else:
                    self.table[i][j] = (
                        old_row[j]
                        - coefficient
                        * old_pivot_row[j]
                        / pivot_element
                    )

            self.table[i][-1] = (
                old_row[-1]
                - coefficient
                * old_pivot_row[-1]
                / pivot_element
            )
        self.table[row] = new_pivot_row
        self.basis[row] = entering_variable
        self.nonbasis[column] = leaving_variable
```

10. Функция simplex

Повторяет симплекс-итерации: выбирает разрешающий столбец и строку, выполняет pivot и выводит новую таблицу. Работа заканчивается после достижения оптимальной таблицы

```
    def simplex(self, phase_name, objective_name):
        step = 1

        while True:
            column = self.find_entering_column()

            if column is None:
                print(
                    f"\n{phase_name}: "
                    "достигнута оптимальная таблица"
                )
                break

            row = self.find_leaving_row(column)

            if row is None:
                raise ValueError(
                    "Целевая функция не ограничена"
                )

            self.pivot(row, column)

            self.print_table(
                f"{phase_name}. "
                f"Таблица после шага {step}",
                objective_name,
            )

            step += 1
```

11. Функция remove_artificial_variables

После первой фазы выводит искусственные переменные из базиса и удаляет их столбцы. Если искусственная переменная остаётся базисной с ненулевой правой частью, задача не имеет допустимых решений

```
def remove_artificial_variables(self):
    i = 0

    while i < len(self.basis):
        if self.basis[i] not in self.artificial:
            i += 1
            continue

        if self.table[i][-1] != 0:
            raise ValueError(
                "У исходной задачи нет "
                "допустимых решений"
            )

        pivot_column = None

        for j, variable in enumerate(self.nonbasis):
            if (
                variable not in self.artificial
                and self.table[i][j] != 0
            ):
                pivot_column = j
                break

        if pivot_column is not None:
            self.pivot(i, pivot_column)
            i += 1

        else:
            del self.table[i]
            del self.basis[i]
```

```
        keep_columns = [
            i
            for i, variable in enumerate(self.nonbasis)
            if variable not in self.artificial
        ]

        new_nonbasis = [
            self.nonbasis[i]
            for i in keep_columns
        ]

        new_table = []

        for i in range(len(self.basis)):
            row = [
                self.table[i][j]
                for j in keep_columns
            ]

            row.append(self.table[i][-1])
            new_table.append(row)

        self.nonbasis = new_nonbasis
        self.table = new_table
```

12. Функция build_original_objective

После удаления искусственных переменных восстанавливает строку исходной целевой функции для второй фазы. Учитывает направление max или min и текущий базис

```
    def build_original_objective(self):
        costs = {}

        if self.sense == "max":
            multiplier = Fraction(1)
        else:
            multiplier = Fraction(-1)

        for i, variable in enumerate(self.variables):
            costs[variable] = (
                multiplier * self.objective[i]
            )

        for variable in self.basis + self.nonbasis:
            if variable not in costs:
                costs[variable] = Fraction(0)

        objective_row = []

        for j, variable in enumerate(self.nonbasis):
            coefficient = -costs[variable]

            for i, basic_variable in enumerate(self.basis):
                coefficient += (
                    costs[basic_variable]
                    * self.table[i][j]
                )

            objective_row.append(coefficient)

        objective_value = Fraction(0)

        for i, basic_variable in enumerate(self.basis):
            objective_value += (
                costs[basic_variable]
                * self.table[i][-1]
            )

        objective_row.append(objective_value)

        self.table.append(objective_row)
```

13. Функции restore_solution и get_solution

Восстанавливают значения исходных переменных после преобразований и вычисляют итоговое значение исходной целевой функции



```
    def restore_solution(self, temp_solution):
        solution = {}

        for i in range(self.variable_count):
            name = f"x{i + 1}"

            value = Fraction(0)

            for new_name, multiplier in self.var_map[name]:
                value += (
                    multiplier
                    * temp_solution[new_name]
                )

            solution[name] = value

        return solution

    def get_solution(self):
        temp_solution = self.get_temp_solution()

        solution = self.restore_solution(
            temp_solution
        )

        objective_value = sum(
            self.input_objective[i]
            * solution[f"x{i + 1}"]
            for i in range(self.variable_count)
        )

        return solution, objective_value
```

14. Функция print_preparation

Выводит выполненные перед решением преобразования: замену свободных переменных и ограничения после обработки отрицательных правых частей



```
    def print_preparation(self):
        print("\nПРЕОБРАЗОВАНИЕ ИСХОДНОЙ ЗАДАЧИ")

        for i in range(self.variable_count):
            name = f"x{i + 1}"

            if self.variable_signs[i] == "free":
                replacement = self.var_map[name]

                print(
                    f"{name} является свободной: "
                    f"{name} = "
                    f"{replacement[0][0]} - "
                    f"{replacement[1][0]}"
                )

        print(
            "\nПосле обработки свободных переменных "
            "и отрицательных правых частей:"
        )

        for coefficients, sign, rhs in self.constraints:
            parts = []

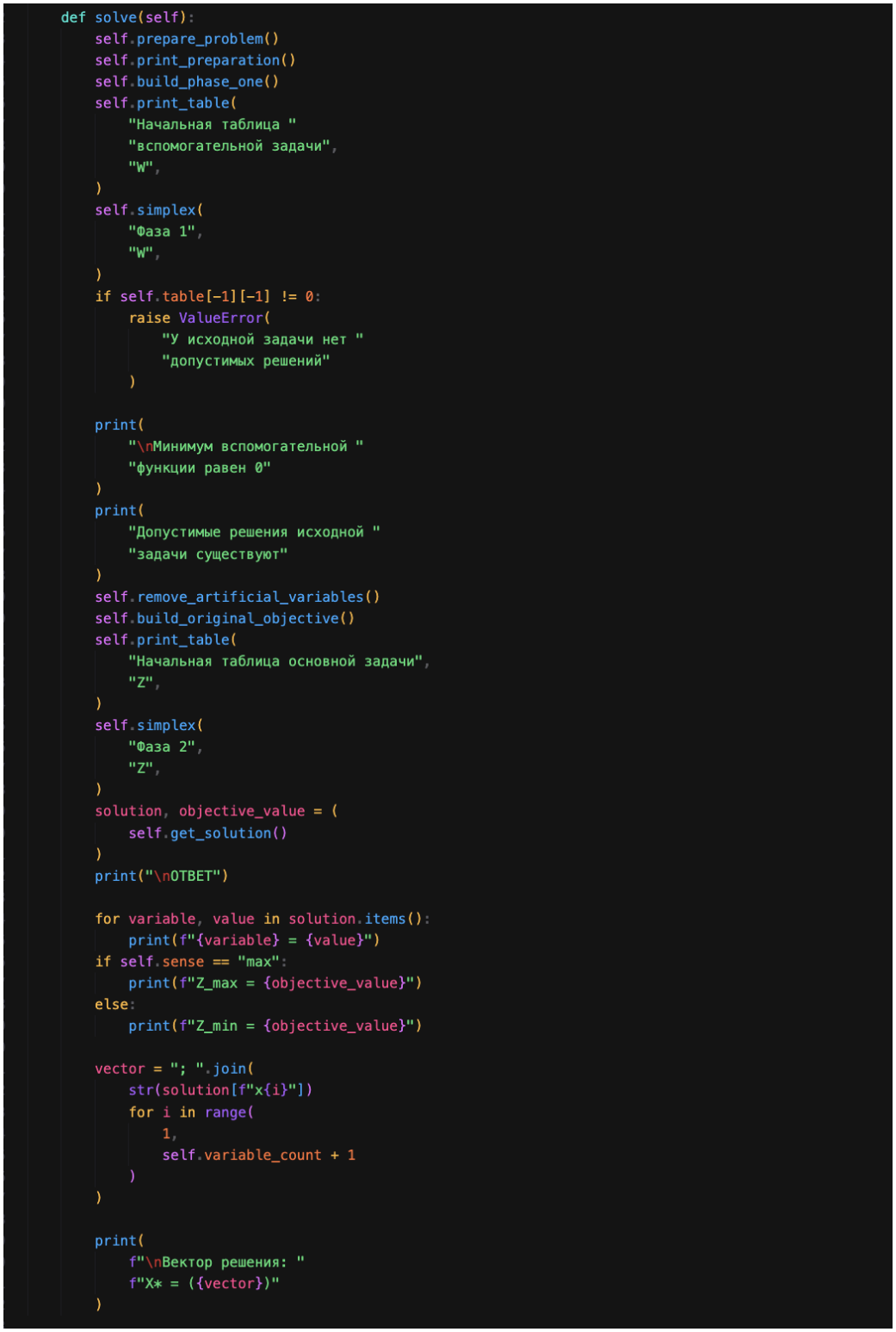
            for coefficient, variable in zip(
                coefficients,
                self.variables,
            ):
                if coefficient != 0:
                    parts.append(
                        f"{coefficient}*{variable}"
                    )

            left = " + ".join(parts)

            print(
                f"{left} {sign} {rhs}"
            )
```

15. Функция solve

Запускает решение целиком: подготовку задачи, первую фазу, проверку допустимости, удаление искусственных переменных, построение исходной целевой функции, вторую фазу и вывод ответа



```
    def solve(self):
        self.prepare_problem()
        self.print_preparation()
        self.build_phase_one()
        self.print_table(
            "Начальная таблица "
            "вспомогательной задачи",
            "W",
        )
        self.simplex(
            "Фаза 1",
            "W",
        )
        if self.table[-1][-1] != 0:
            raise ValueError(
                "У исходной задачи нет "
                "допустимых решений"
            )

        print(
            "\nМинимум вспомогательной "
            "функции равен 0"
        )
        print(
            "Допустимые решения исходной "
            "задачи существуют"
        )
        self.remove_artificial_variables()
        self.build_original_objective()
        self.print_table(
            "Начальная таблица основной задачи",
            "Z",
        )
        self.simplex(
            "Фаза 2",
            "Z",
        )
        solution, objective_value = (
            self.get_solution()
        )
        print("\nОТВЕТ")

        for variable, value in solution.items():
            print(f"{variable} = {value}")
        if self.sense == "max":
            print(f"Z_max = {objective_value}")
        else:
            print(f"Z_min = {objective_value}")

        vector = "; ".join(
            str(solution[f"x{i}"])
            for i in range(
                1,
                self.variable_count + 1
            )
        )

        print(
            f"\nВектор решения: "
            f"X* = ({vector})"
        )
```

16. Исходные данные варианта

Задаются коэффициенты целевой функции, три ограничения, типы переменных и направление максимизации. После этого создаётся SimplexSolver и вызывается solve()

```python
objective = [1, 3, 2, 1]

constraints = [
    ([1, 1, 0, 2], "<=", 8),
    ([0, 1, 1, 1], "=", 6),
    ([2, 0, 1, 0], ">=", 2),
]


variable_signs = [
    "nonnegative",
    "nonnegative",
    "nonnegative",
    "nonnegative",
]

solver = SimplexSolver(
    objective,
    constraints,
    sense="max",
    variable_signs=variable_signs,
)

solver.solve()
```

17. Результат работы программы

В результате выполнения программы получено оптимальное решение $X^* = (2; 6; 0; 0)$ и максимальное значение целевой функции $Z_{\max} = 20$. Результат совпадает с решением,

полученным вручную

```
mariafedorova@MacBook-Pro-Maria-4 методы % python3 simplex.py

Начальная таблица вспомогательной задачи
          x1      x2      x3      x4      x5      x6       b
    x7     1       1       0       2       1       0       8
    x8     0       1       1       1       0       0       6
    x9     2       0       1       0       0      -1       2
     W    -3      -2      -2      -3      -1       1     -16

Разрешающий столбец: x1
Разрешающая строка: x9

Входит в базис: x1
Выходит из базиса: x9
Разрешающий элемент: 2

Фаза 1. Таблица после шага 1
          x9      x2      x3      x4      x5      x6       b
    x7  -1/2     1.0    -1/2     2.0     1.0    1/2     7.0
    x8   0.0     1.0     1.0     1.0     0.0    0.0     6.0
    x1   1/2     0.0     1/2     0.0     0.0   -1/2     1.0
     W   3/2    -2.0    -1/2    -3.0    -1.0   -1/2   -13.0

Разрешающий столбец: x4
Разрешающая строка: x7

Входит в базис: x4
Выходит из базиса: x7
Разрешающий элемент: 2

Фаза 1. Таблица после шага 2
          x9      x2      x3      x7      x5      x6       b
    x4  -1/4     1/2    -1/4     1/2     1/2    1/4     7/2
    x8   1/4     1/2     5/4    -1/2    -1/2   -1/4     5/2
    x1   1/2     0.0     1/2     0.0     0.0   -1/2     1.0
     W   3/4    -1/2    -5/4     3/2     1/2    1/4    -5/2

Разрешающий столбец: x3
Разрешающая строка: x8

Входит в базис: x3
Выходит из базиса: x8
Разрешающий элемент: 5/4

Фаза 1. Таблица после шага 3
          x9      x2      x8      x7      x5      x6       b
    x4  -1/5     3/5     1/5     2/5     2/5     1/5     4.0
    x3   1/5     2/5     4/5    -2/5    -2/5    -1/5     2.0
    x1   2/5    -1/5    -2/5     1/5     1/5    -2/5     0.0
     W   1.0     0.0     1.0     1.0     0.0     0.0     0.0

Фаза 1: достигнута оптимальная таблица

Минимум вспомогательной функции равен 0
Допустимые решения исходной задачи существуют

Начальная таблица основной задачи
          x2      x5      x6       b
    x4   3/5     2/5     1/5     4.0
    x3   2/5    -2/5    -1/5     2.0
    x1  -1/5     1/5    -2/5     0.0
     Z  -9/5    -1/5    -3/5     8.0

Разрешающий столбец: x2
Разрешающая строка: x3

Входит в базис: x2
Выходит из базиса: x3
Разрешающий элемент: 2/5
```

```
Фаза 2. Таблица после шага 1
           x3      x5      x6       b
    x4    −3/2      1     1/2       1
    x2     5/2     −1    −1/2       5
    x1     1/2      0    −1/2       1
     Z     9/2     −2    −3/2      17

Разрешающий столбец: x5
Разрешающая строка: x4

Входит в базис: x5
Выходит из базиса: x4
Разрешающий элемент: 1

Фаза 2. Таблица после шага 2
           x3      x4      x6       b
    x5    −3/2      1     1/2       1
    x2       1      1       0       6
    x1     1/2      0    −1/2       1
     Z     3/2      2    −1/2      19

Разрешающий столбец: x6
Разрешающая строка: x5

Входит в базис: x6
Выходит из базиса: x5
Разрешающий элемент: 1/2

Фаза 2. Таблица после шага 3
           x3      x4      x5       b
    x6      −3       2       2       2
    x2       1       1       0       6
    x1      −1       1       1       2
     Z       0       3       1      20

Фаза 2: достигнута оптимальная таблица

ОТВЕТ
x1 = 2
x2 = 6
x3 = 0
x4 = 0
Z_max = 20

Вектор решения: X* = (2; 6; 0; 0)
```